1 Polar Vortex Trajectory Generation and Data Preparation

This section generates particle trajectories from MATLAB data and prepares the spacetime data used for coherent structure analysis.

1.1 Velocity Field and Dataset

The velocity field is obtained from the atmospheric data. Please refer to the following publications:

- https://npg.copernicus.org/articles/24/661/2017/
- https://pubs.aip.org/cha/article/20/4/043116/152085/Transport-in-time-dependent-dynamical-systems

The trajectories can be loaded from `trajectories_cache.npz` for computations. The flow is represented as a time-dependent velocity field.

$$\frac{dx}{dt} = v(t, x),$$

where $x = (x_1, x_2) \in \mathbb{R}^2$.

1.2 Initial Conditions and Trajectory Integration

Initial particle positions are generated on a uniform Cartesian grid

$$[-7000, 7000] \times [-7000, 7000] \text{ km}.$$

Trajectories are computed over the interval

$$[t_{\text{start}}, t_{\text{end}}],$$

with

$$t_{\text{start}} = 1, \qquad t_{\text{end}} = 120.$$

The system

$$\frac{dX_i(t)}{dt} = v(t, X_i(t))$$

is solved numerically for every particle. The trajectory data are organized as

$$\text{SpacePointsarray} \in \mathbb{R}^{2\times N\times T},$$

where

- the first dimension corresponds to spatial coordinates,
- the second dimension indexes particles,
- the third dimension indexes time.

Computed trajectories are cached in `trajectories_cache.npz` for future runs.

2 Parameter Selection

2.1 Temporal Coupling Parameter Selection

This section computes the temporal coupling parameter used in the inflated dynamic Laplacian.

Since the initial domain is approximately disk-shaped and remains so under the flow, $a_{\min}$ is obtained by equating the leading Dirichlet-Laplacian eigenvalue of a disk of radius r_0, giving the lower bound

$$a_{\min} = \frac{j_{0,1}\,\tau}{\pi r_0}.$$

where

$$\tau$$

is the total integration time and

$$r_0$$

is the characteristic domain radius.

For non-Dirichlet boundary conditions $j_{0,1} \approx 2.4048$ is replaced by $j_{1,1} \approx 1.8412$, the first root of J_1.

2.2 Bandwidth Estimation

For each time slice, nearest-neighbour distances are computed using a KD-tree.

Let

$$d_i$$

denote the nearest-neighbour distance of particle i.

The bandwidth estimate is obtained from

$$\varepsilon = c\,\frac{1}{T}\sum_{k=1}^{T}\overline{d_k^2},$$

where c is a user-selected scale factor.

3 Diffusion Maps and Spacetime Operators

3.1 Diffusion Kernel

For a point cloud $\{x_i\}$, the kernel matrix is defined by

$$K_{ij} = \exp\!\left(-\frac{\|x_i - x_j\|^2}{4\varepsilon}\right).$$

Only nearby neighbours are retained, producing a sparse matrix representation.

The kernel matrix is normalized to obtain a diffusion operator. Density normalization is applied first, followed by row normalization to produce a Markov matrix

$$P_x.$$

3.2 Spatial Diffusion Operator

For every time slice, a diffusion maps operator is constructed.

The resulting operators are assembled into a block-diagonal matrix

$$P_x.$$

Each block corresponds to one temporal snapshot.

3.3 Temporal Laplacian

Temporal diffusion is represented by a finite-difference approximation of

$$L_t \approx \frac{\partial^2}{\partial t^2}.$$

The associated temporal diffusion operator is generated from L_t as

$$P_t^{1/2} = \exp\Big(\frac{\varepsilon}{2} L_t\Big).$$

3.4 Strang Splitting Operator

The spacetime diffusion operator is constructed using Strang splitting:

$$P_a = P_t^{1/2} P_x P_t^{1/2}.$$

This formulation avoids explicit construction of the full spacetime matrix and enables efficient matrix-vector products.

4 Sparse Eigenbasis Approximation (SEBA)

SEBA is used to obtain sparse representations of the leading eigenvectors. Given an eigenvector matrix

$$V \in \mathbb{R}^{p \times r},$$

a QR decomposition is first applied, $V = QR$. A soft-thresholding operation is then applied,

$$s_i = \operatorname{sign}(z_i) \max(|z_i| - \mu, 0), \qquad \mu = \frac{0.99}{\sqrt{p}}.$$

After thresholding, a polar decomposition step updates the rotation matrix using singular value decomposition (SVD). The iteration continues until convergence,

$$\|R_{\text{new}} - R\| < \text{tol}.$$

Finally, the sparse modes are normalized and sorted to produce coherent structure indicators, promoting sparsity and enhancing interpretability of coherent structures.

5 Spectral Decomposition and Time Averaged Spatial Operator

5.1 Eigenvalue Problem

The spacetime diffusion operator P_a is represented as a linear operator.

The leading eigenpairs

$$(\nu_k, v_k)$$

are computed using sparse eigensolvers.

Spatial and temporal decay rates are estimated from the action of the spatial and temporal diffusion operators on the computed eigenmodes.

5.2 Time Averaged Spatial Operator

A time-averaged spatial diffusion operator

$$\overline{P_x}$$

is constructed from the collection of spatial diffusion matrices.

The dominant eigenvectors of

$$\overline{P_x}$$

provide coherent structures that persist over the full observation interval.

6 Visualization and Analysis

6.1 Eigenvalue Spectrum

Eigenvalues are transformed according to

$$\Lambda_k = \frac{\log(\nu_k)}{\varepsilon}.$$

The resulting spectrum is used to identify dominant coherent structures.

6.2 Animations

Spatial Eigenmode Animated visualizations are generated for the leading spacetime eigenmodes. Each animation displays the temporal evolution of coherent structures across the polar vortex domain.

Time-Averaged Coherent Sets Animations are also generated for eigenvectors of the averaged spatial operator. These visualizations highlight persistent coherent regions.

Thresholding of dominant eigenvectors is used to isolate regions associated with the vortex core. This way the Polar Vortex Core is Identified. Snapshots are produced at selected times to illustrate coherent-set evolution.

The geometric deformation of the coherent vortex region is tracked over time to track Vortex Shape Deformation .The resulting figures illustrate stretching, contraction, and shape variation of the vortex boundary.

7 Main Execution Script

The script

`run_all.py`

orchestrates the complete workflow:

- loading or computing trajectories,

- constructing the spacetime dataset,
- computing the temporal coupling parameter,
- identifying boundary points,
- constructing diffusion operators,
- applying SEBA to the leading eigenvectors,
- solving the spacetime eigenproblem,
- computing averaged spatial operators,
- generating eigenvalue spectra,
- producing coherent-set animations,
- creating vortex-core visualizations.

All outputs are saved in the directory results.